

Figure 7. **Two stages of circuit MLPs.** MLPs 14-18's outputs are well modeled by $\text{helix}(a+b)$, suggesting that they build this helical representation, while MLPs 19-27 higher DE/TE ratio implies they output the answer to model logits.

fit and more MLP output affects answer logits directly. We interpret this as MLPs 14-18 "building" the $a+b$ helix, which MLPs 19-27 translate to the answer token $a+b$.

However, our MLP analysis has focused solely on MLP outputs. To demonstrate the Clock algorithm conclusively, we must look at MLP inputs. Recall that GPT-J uses a simple MLP: $\text{MLP}(x) = \sigma(xW_{\text{up}})W_{\text{down}}$. x is a vector of size $(4096,)$ representing the residual stream, and W_{up} is a $(4096, 16384)$ projection matrix. The input to the MLP is thus the 16384 dimensional xW_{up}. We denote the nth dimension of the MLP input as the nth neuron preactivation, and move to analyze these preactivations.

5.4. Zooming in on Neurons

Activation patching the $27 * 16384$ neurons in GPT-J is prohibitively expensive, so we instead use the technique of attribution patching to approximate the total effect of each neuron using its gradient (see Kramár et al. (2024)). We find that using just 1% of the neurons in GPT-J and mean ablating the rest allows for the successful completion of 80% of prompts (see Appendix D.2). Thus, we focus our analysis on this sparse set of $k = 4587$ neurons.

5.4.1. Modeling Neuron Preactivations

For a prompt $a+b$, we denote the preactivation of the nth neuron in layer l as $N_n^l(a,b)$. When we plot a heatmap of $N_n^l(a,b)$ for top neurons in Fig. 8, we see that their preactivations are periodic in a, b, and $a+b$. When we Fourier decompose the preactivations as a function of $a+b$, we find that the most common periods are $T = [2, 5, 10, 100]$, matching those used in our helix parameterization (Appendix D.2). This is sensible, as the nth neuron in a layer applies W_{up}^n of shape $(4096,)$ to the residual stream, which we have effectively modeled as a $\text{helix}(a, b, a+b)$. Subsequently, we model the preactivation of each top neuron as

$$N_n^l(a,b) = \sum_{t=a,b,a+b} c_t t + \sum_{T=[2,5,10,100]} c_{Tt} \cos\left(\frac{2\pi}{T}(t - d_{Tt})\right) \tag{3}$$

For each neuron preactivation, we fit the parameters c and d in Eq. 3 using gradient descent (see Appendix D.2 for details). In Fig. 8, we show the highest magnitude fit component for a selection of top neurons.

We evaluate our fit of the top k neurons by patching them into the model, mean ablating all other neurons, and measuring the resulting accuracy of the model. In Fig. 9, we see that our neuron fits provide roughly 75% of the performance of using the actual neuron preactivations. Thus, these neurons are well modeled as reading from the helix.

5.4.2. Understanding MLP Inputs

We use our understanding of neuron preactivations to draw conclusions about MLP inputs. To do so, we first path patch each of the top k neurons to find their direct effect and calculate their DE/TE ratio. For each neuron, we calculate the fraction of their fit that $\text{helix}(a+b)$ explains, which we approximate by dividing the magnitude of $c_{T,a+b}$ terms by the total magnitude of c_{Tt} terms in Eq. 3. For each circuit MLP, we calculate the mean of both of these quantities across top neurons, and visualize them in Fig. 10.

Once again, we see a split at layer 19, where earlier neurons' preactivation fits rely on a, b terms, while later neurons use $a+b$ terms and write to logits. Since the neuron preactivations represent what each MLP is "reading" from, we combine this result with our evidence from Section 5.3 to summarize the role of MLPs in addition.

- MLPs 14-18 primarily read from the a, b helices to create the $a+b$ helix for downstream processing.
- MLPs 19-27 primarily read from the $a+b$ helix to write the answer to model logits.

Thus, we conclude our case that LLMs use the Clock algorithm to do addition, with a deep investigation into how GPT-J implements this algorithm.

5.5. Limitations of Our Understanding

There are several aspects of LLM addition we still do not understand. Most notably, while we provide compelling evidence that key components create $\text{helix}(a+b)$ from $\text{helix}(a, b)$, we do not know the exact mechanism they use to do so. We hypothesize that LLMs use trigonometric identities like $\cos(a+b) = \cos(a)\cos(b) - \sin(a)\sin(b)$ to create $\text{helix}(a+b)$. However, like the originator of the Clock algorithm Nanda et al. (2023a), we are unable to isolate this computation in the model. This is unsurprising, as

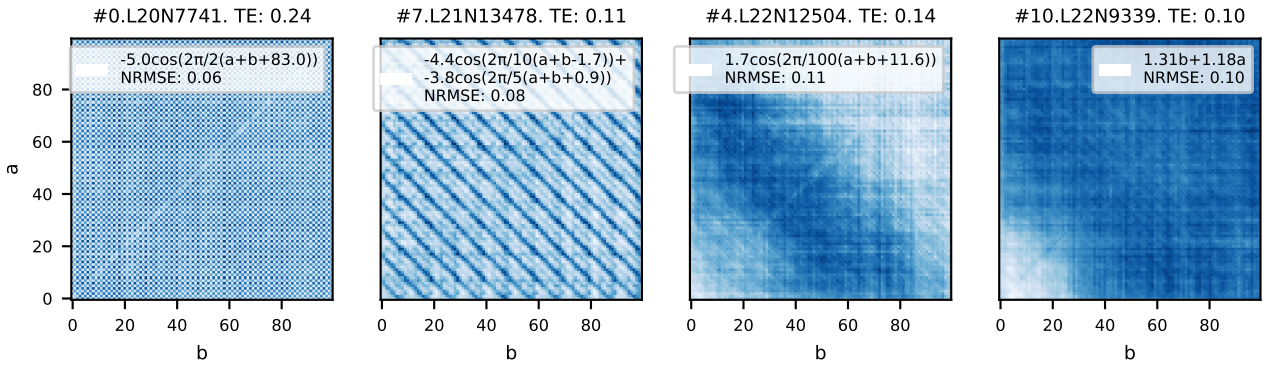


Figure 8. **Neuron preactivations and fits.** We visualize the preactivations $N_n^l(a, b)$ for four top neurons. Each neuron has clear periodicity in its preactivations, which we model using a helix inspired functional form.

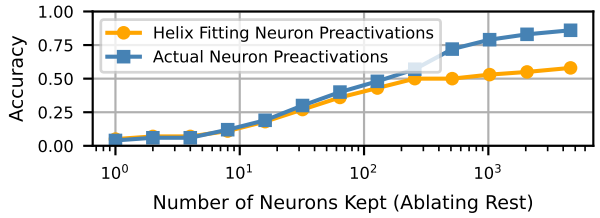


Figure 9. **Evaluating neuron fits.** Patching in our fitted preactivations for the top k neurons is roughly as effective as patching in their actual preactivations and ablating all other neurons.

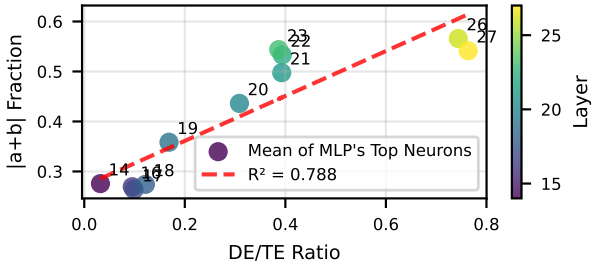


Figure 10. **Neuron trends.** Neurons in MLPs 14-18 primarily read from the a, b helices, while MLPs 19-27 primarily read from the $a + b$ helix and write to logits.

there is a large solution space for how models choose to implement low-level details of algorithms. For example, Yip et al. (2024) finds that in Zhong et al. (2023)'s "Pizza" algorithm for modular addition, MLPs in one layer transformers implement numerical integration techniques to transform $\cos(\frac{k}{2}(a+b))$ to $\cos(k(a+b))$.

The mere existence of the Pizza algorithm demonstrates that even one layer transformers have a complex solution space. Thus, even if the Clock algorithm is used by LLMs, it could be one method of an ensemble. We see evidence of this in Appendix D, in that $\text{helix}(a+b)$ is less causally implicated for Llama3.1-8B than other models, which we hypothesize is due to its use of gated MLPs. Additionally, other models must necessarily use modified algorithms for addition because of different tokenization schemes. For example, Gemma-2-9B (Riviere et al., 2024) tokenizes each digit of a number separately and must use additional algorithms to collate digit tokens. Additionally, at different scales LLMs potentially learn different algorithms, providing another reason to be skeptical that the Clock algorithm is the one and only explanation for LLM addition.

6. Conclusion

We find that three mid-sized LLMs represent numbers as generalized helices and manipulate them using the interpretable Clock algorithm to compute addition. While LLMs could do addition linearly, we conjecture that LLMs use the Clock algorithm to improve accuracy, analogous to humans using decimal digits (which are a generalized helix with $T = [10, 100, \ldots]$) for addition rather than slide rules. In Appendix E, we present preliminary results that GPT-J would be considerably less accurate on "linear addition" due to noise in its linear representations. Future work could analyze if LLMs have internal error-correcting codes for addition like the grid cells presented in Zlokapa et al. (2024).

The use of the Clock algorithm provides striking evidence that LLMs trained on general text naturally learn to implement complex mathematical algorithms. Understanding LLM algorithms is important for safe AI and can also provide valuable insight into model errors, as shown in Appendix F. We hope that this work inspires additional investigations into LLM mathematical capabilities, especially as addition is implicit to many reasoning problems.

Acknowledgments

We thank Josh Engels for participating in extensive conversations throughout the project. We also thank Vedang Lad, Neel Nanda, Ziming Liu, David Baek, and Eric Michaud for their helpful suggestions. This work is supported by the Rothberg Family Fund for Cognitive Science and IAIFI through NSF grant PHY-2019786.

References

Ahn, J., Verma, R., Lou, R., Liu, D., Zhang, R., and Yin, W. Large language models for mathematical reasoning: Progresses and challenges, 2024. URL https://arxiv.org/abs/2402.00157.

Biderman, S., Schoelkopf, H., Anthony, Q. G., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.

Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., McLean, B., Burke, J. E., Hume, T., Carter, S., Henighan, T., and Olah, C. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. https://transformer-circuits.pub/2023/monosemantic-features/index.html.

Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., Grosse, R., McCandlish, S., Kaplan, J., Amodei, D., Wattenberg, M., and Olah, C. Toy models of superposition. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.

Engels, J., Michaud, E. J., Liao, I., Gurnee, W., and Tegmark, M. Not all language model features are linear, 2024. URL https://arxiv.org/abs/2405.14860.

Fiotto-Kaufman, J., Loftus, A. R., Todd, E., Brinkmann, J., Juang, C., Pal, K., Rager, C., Mueller, A., Marks, S., Sharma, A. S., Lucchetti, F., Ripa, M., Belfki, A., Prakash, N., Multani, S., Brodley, C., Guha, A., Bell, J., Wallace, B., and Bau, D. Nnsight and ndif: Democratizing access to foundation model internals. 2024. URL https://arxiv.org/abs/2407.14561.

F.R.S., K. P. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11): 559–572, 1901. doi: 10.1080/14786440109462720.

Gao, L., la Tour, T. D., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., and Wu, J. Scaling and evaluating sparse autoencoders, 2024. URL https://arxiv.org/abs/2406.04093.

Glazer, E., Erdil, E., Besiroglu, T., Chicharro, D., Chen, E., Gunning, A., Olsson, C. F., Denain, J.-S., Ho, A., de Oliveira Santos, E., Järviniemi, O., Barnett, M., Sandler, R., Vrzala, M., Sevilla, J., Ren, Q., Pratt, E., Levine, L., Barkley, G., Stewart, N., Grechuk, B., Grechuk, T., Enugandla, S. V., and Wildon, M. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai, 2024. URL https://arxiv.org/abs/2411.04872.

Goldowsky-Dill, N., MacLeod, C., Sato, L., and Arora, A. Localizing model behavior with path patching, 2023. URL https://arxiv.org/abs/2304.05969.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., et al. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Hanna, M., Liu, O., and Variengien, A. How does GPT-2 compute greater-than?: Interpreting mathematical abilities in a pre-trained language model. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=p4PckNQR8k.

Heimersheim, S. and Nanda, N. How to use and interpret activation patching, 2024. URL https://arxiv.org/abs/2404.15255.

Huben, R., Cunningham, H., Smith, L. R., Ewart, A., and Sharkey, L. Sparse autoencoders find highly interpretable features in language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=F76bwRSLeK.

Kramár, J., Lieberum, T., Shah, R., and Nanda, N. Atp*: An efficient and scalable method for localizing llm behaviour to components, 2024. URL https://arxiv.org/abs/2403.00745.